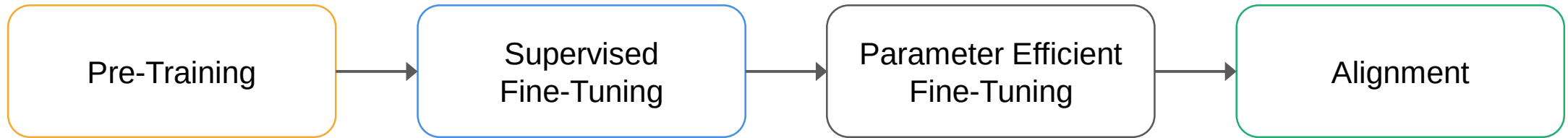


Session 2: LLM Training Overview

Training Stages of LLMs

LIFECYCLE OF A PRODUCTION-READY LLM

The lifecycle of a production-ready LLM is a pipeline of stages



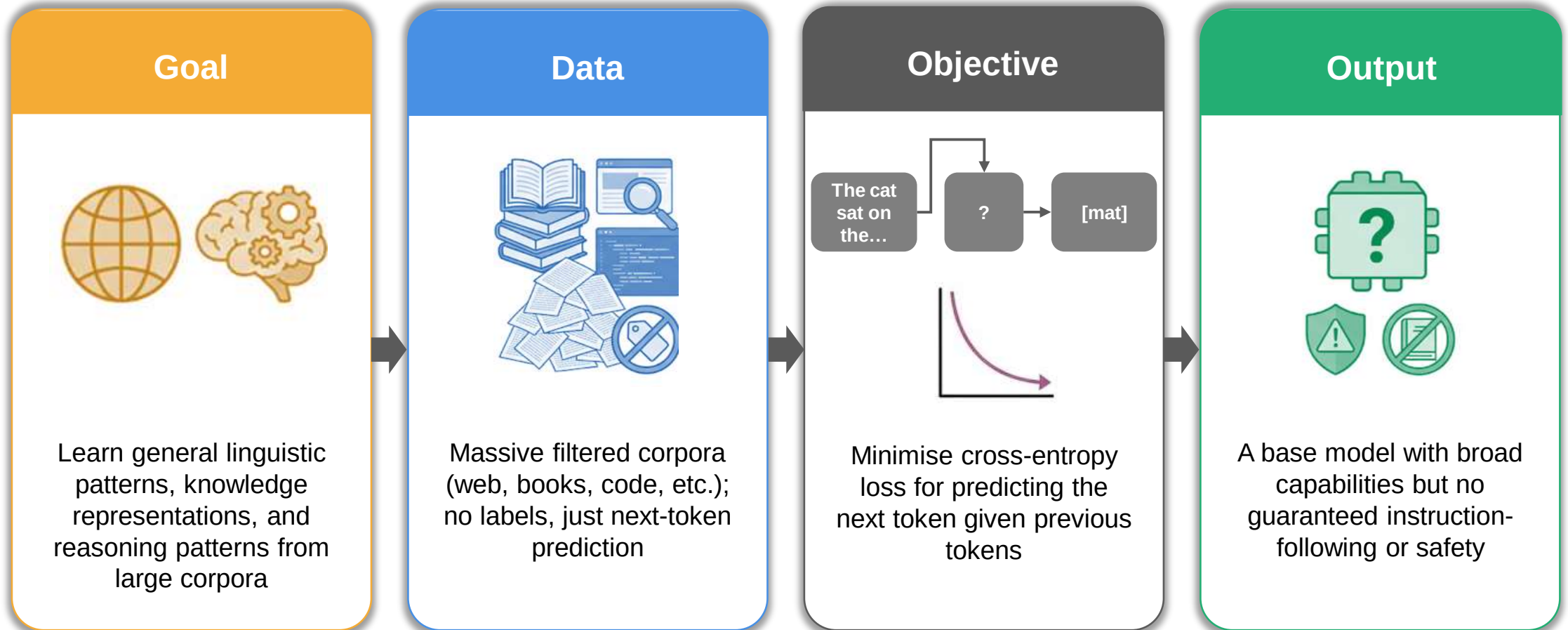
Each stage has a different goal and data type

- Pretraining builds capability
- SFT teaches instruction-following
- PEFT adds efficient task adaptation
- Alignment further optimises model behaviour using human or preference feedback

When we fine-tune or use PEFT, we are plugging into this pipeline after pretraining (and often after SFT)

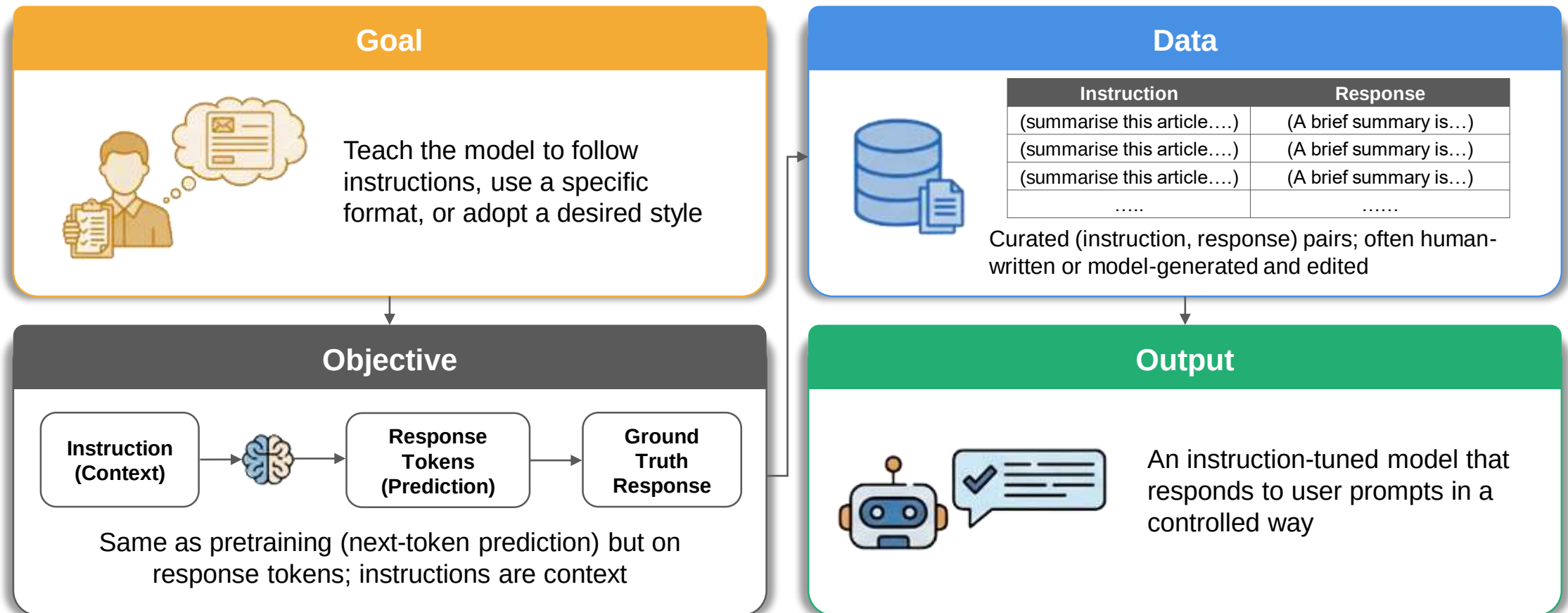
PRE-TRAINING

Pre-training is a crucial stage when developing a production-ready LLM. It is the most time-consuming, expensive, yet critical step that determines the overall quality of the LLM



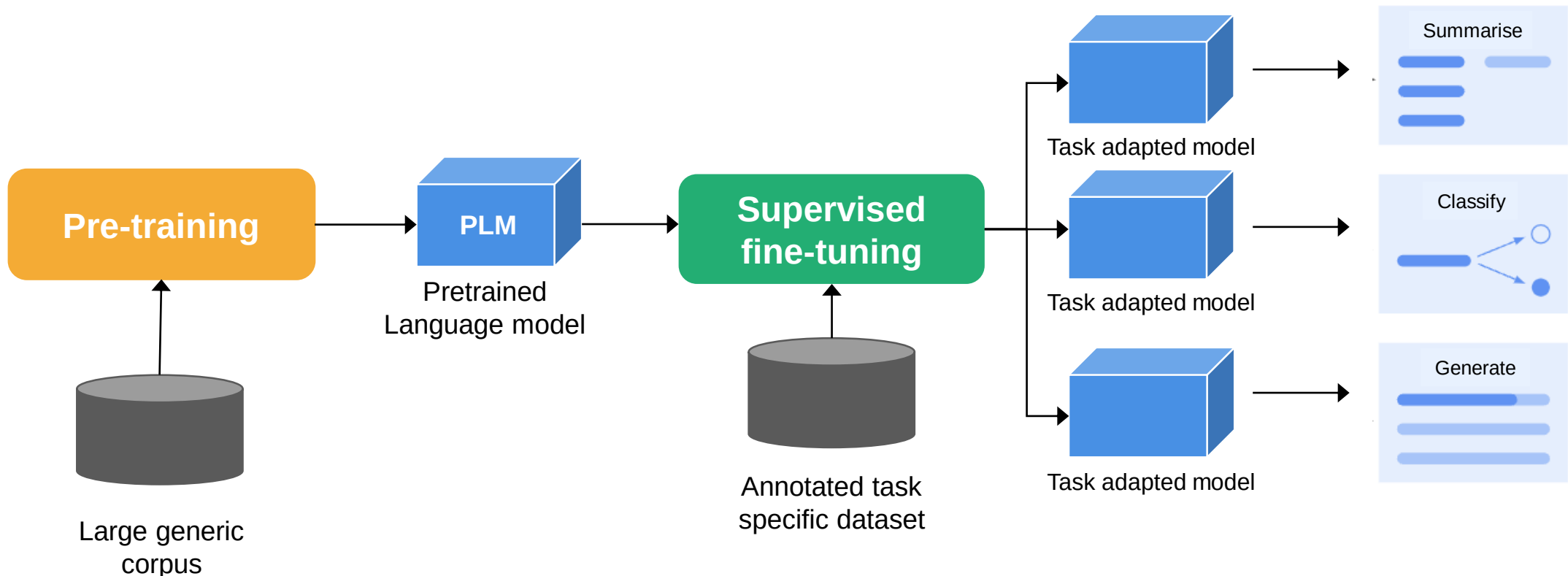
SUPERVISED FINE-TUNING

- Supervised Fine-Tuning (SFT) is the process of further training a pre-trained LLM on a smaller, curated, and labelled dataset to specialise it for specific tasks or domains, such as turning a base model into a chatbot or a medical analyser
- It adjusts the model's weights to follow instructions or adopt specific tones without losing general knowledge



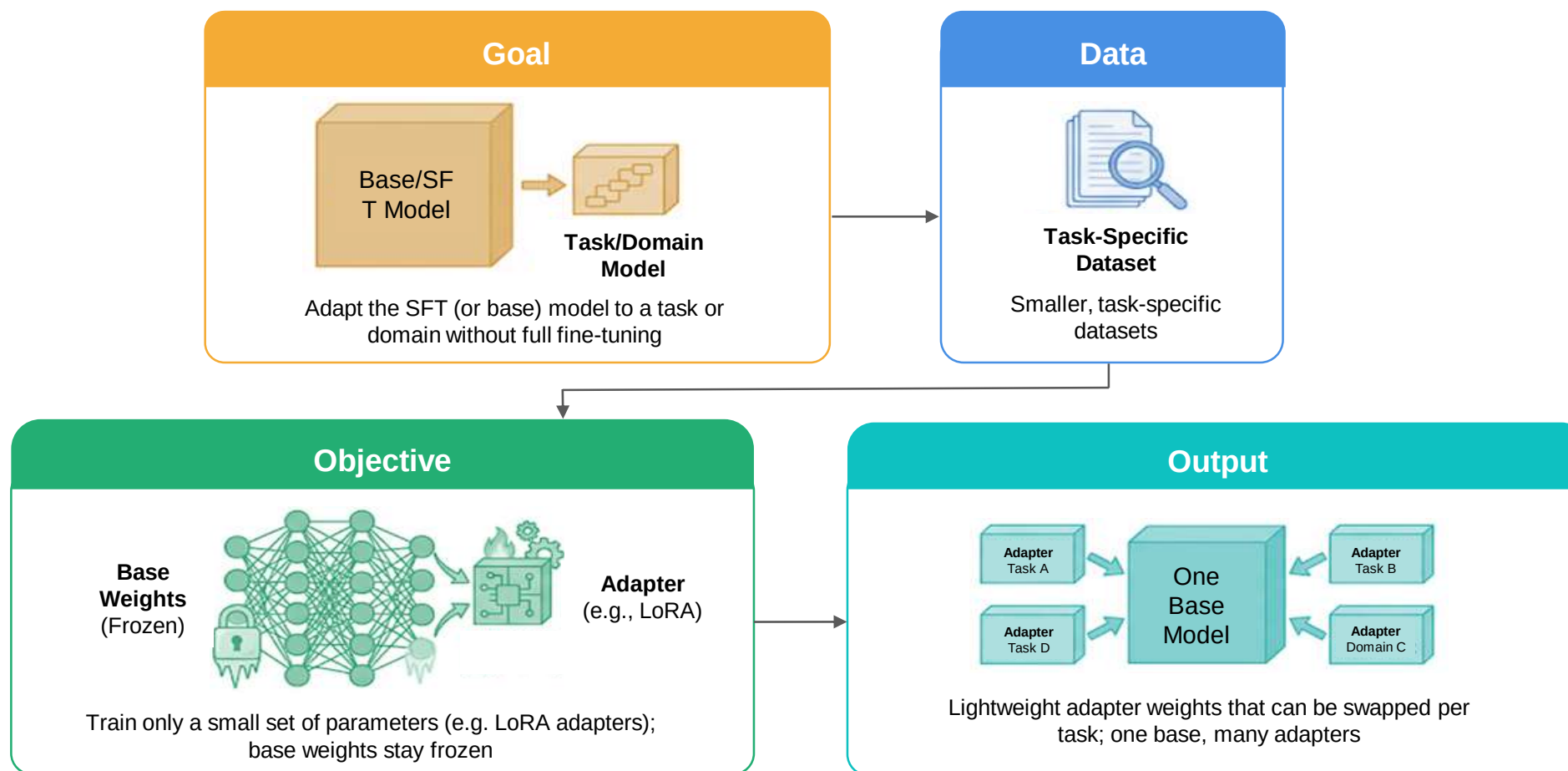
SFT PROCESS

- **Labelled Data:** Unlike pre-training (which uses raw text), SFT uses input-output pairs (e.g., "Summarise this: [Text]" → "[Summary]") validated beforehand
- **Training:** The model learns to map specific inputs to the expected outputs, refining its parameters
- **Instruction Tuning:** A common form of SFT where models are trained to follow instructions



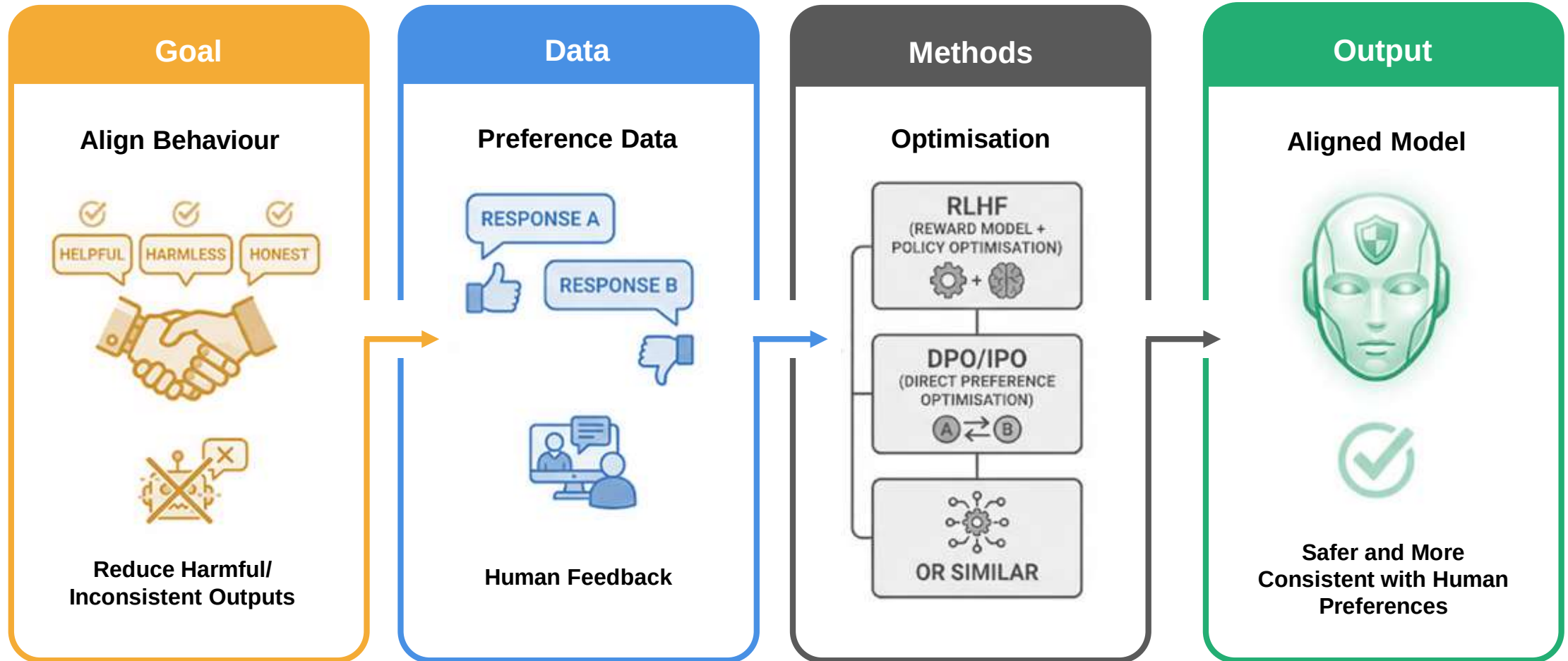
PARAMETER-EFFICIENT FINE-TUNING

- Parameter-Efficient Fine-Tuning (PEFT) is a methodology used to adapt large pre-trained LLMs to specific tasks by updating only a **tiny fraction** of the model's total parameters
- This approach offers a resource-friendly alternative to “full fine-tuning,” which requires retraining all billions of parameters in a model. However, it is an optional step in the entire lifecycle



ALIGNMENT

Alignment trains the model to produce responses preferred by humans, often using preference datasets and optimisation methods such as RLHF or DPO



Training LLMs at Scale

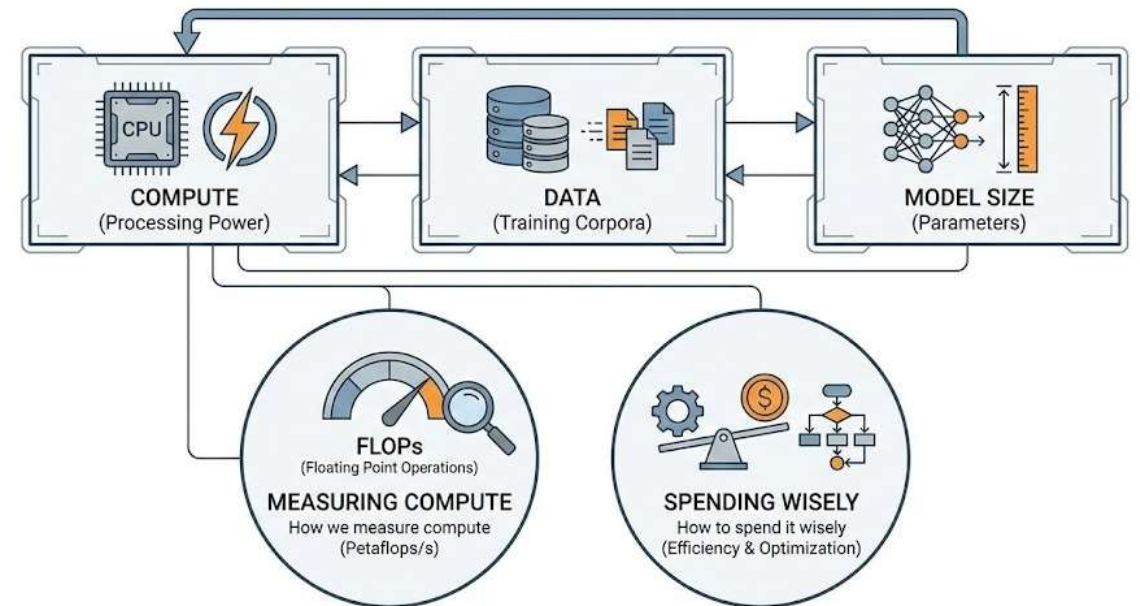
TRAINING LLMs AT SCALE

Training LLMs at scale is governed by

- Compute
- Data
- Model size

When we talk about scale, two key questions are important

- How do we measure compute?
- How to spend it wisely?



COMPUTE: FLOPs vs FLOP/S

FLOPs (Floating Point Operations)

- It is the total “work” done during training
- For example: 10^{23} FLOPs for a 70B model on 1T tokens
- This is the **compute budget**: how much we are willing to spend

FLOP/s (FLOPs per second)

- It is the throughput of an LLM
- FLOP/s measures how fast hardware can perform floating-point operations
- Higher FLOP/s $\Rightarrow$ same FLOPs in less wall-clock time

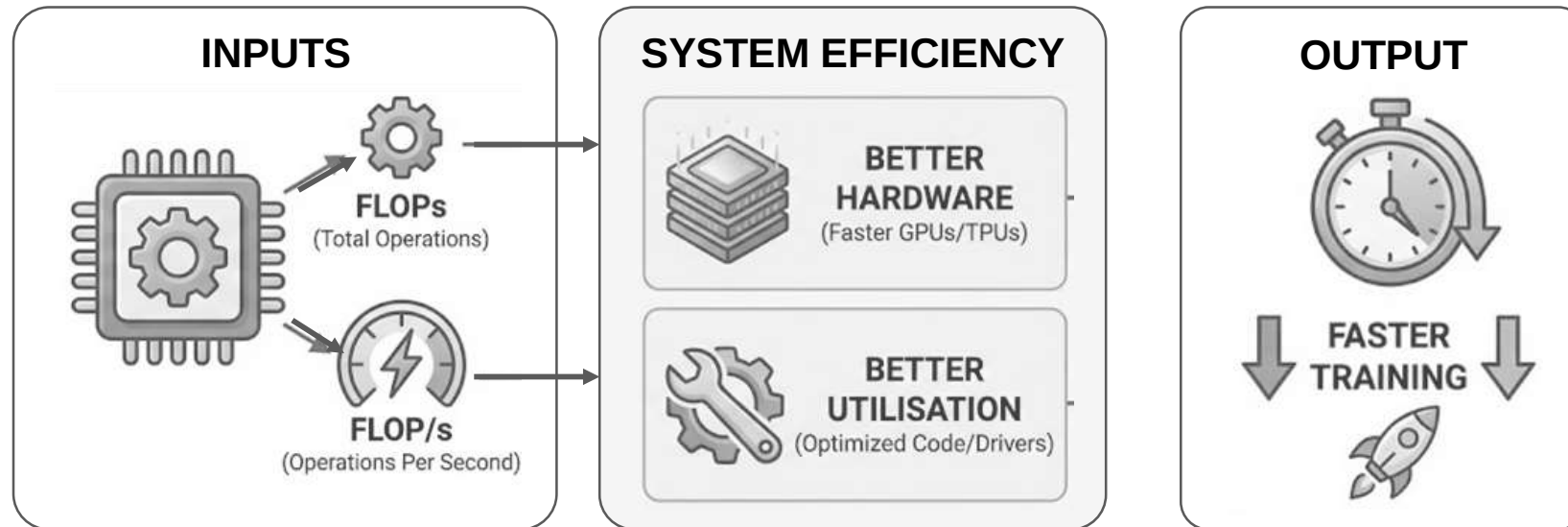
COMPUTE: FLOPs vs FLOP/S

Overall, training time is given by

$$\text{Training time} \approx \text{FLOPs} \div \text{FLOP/s}$$

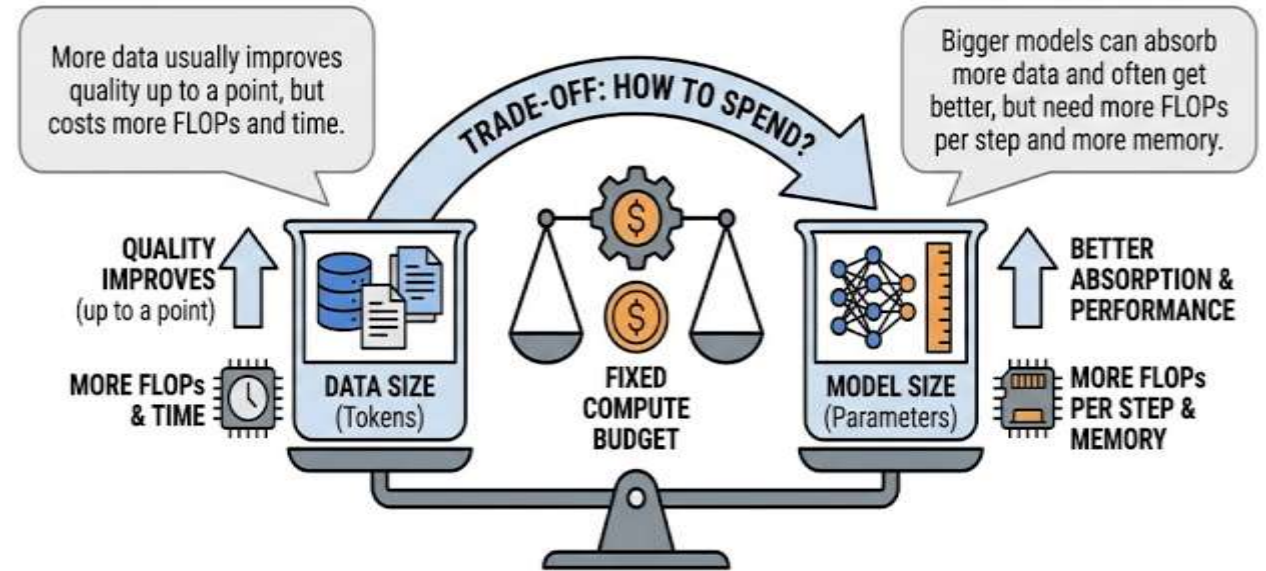
(Ignoring communication and I/O)

This means that the same model and data with better hardware or better utilisation lead to faster training



COMPUTE-DATA TRADE-OFFS

- More **data** (tokens) usually improves quality up to a point but costs more FLOPs and time
- Larger **models** can absorb more data and often get better but need more FLOPs per step and more memory
- Considering a fixed **compute budget**, the trade-off is how much to spend on model size vs data size (and data quality)

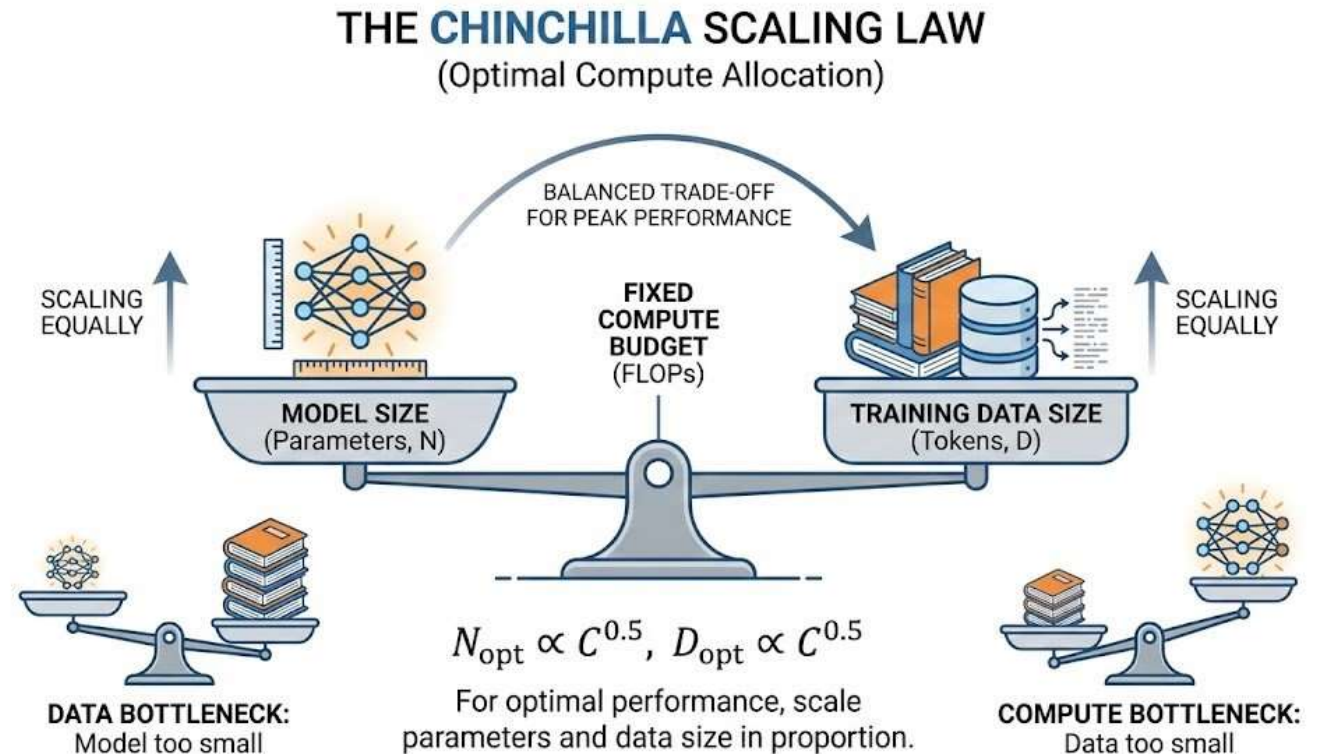


CHINCHILLA LAW

Chinchilla showed that for a given compute budget, **optimal performance** often comes from **smaller models trained on more data** than was common before (e.g. GPT-3-style scaling)

Many large models were under-trained

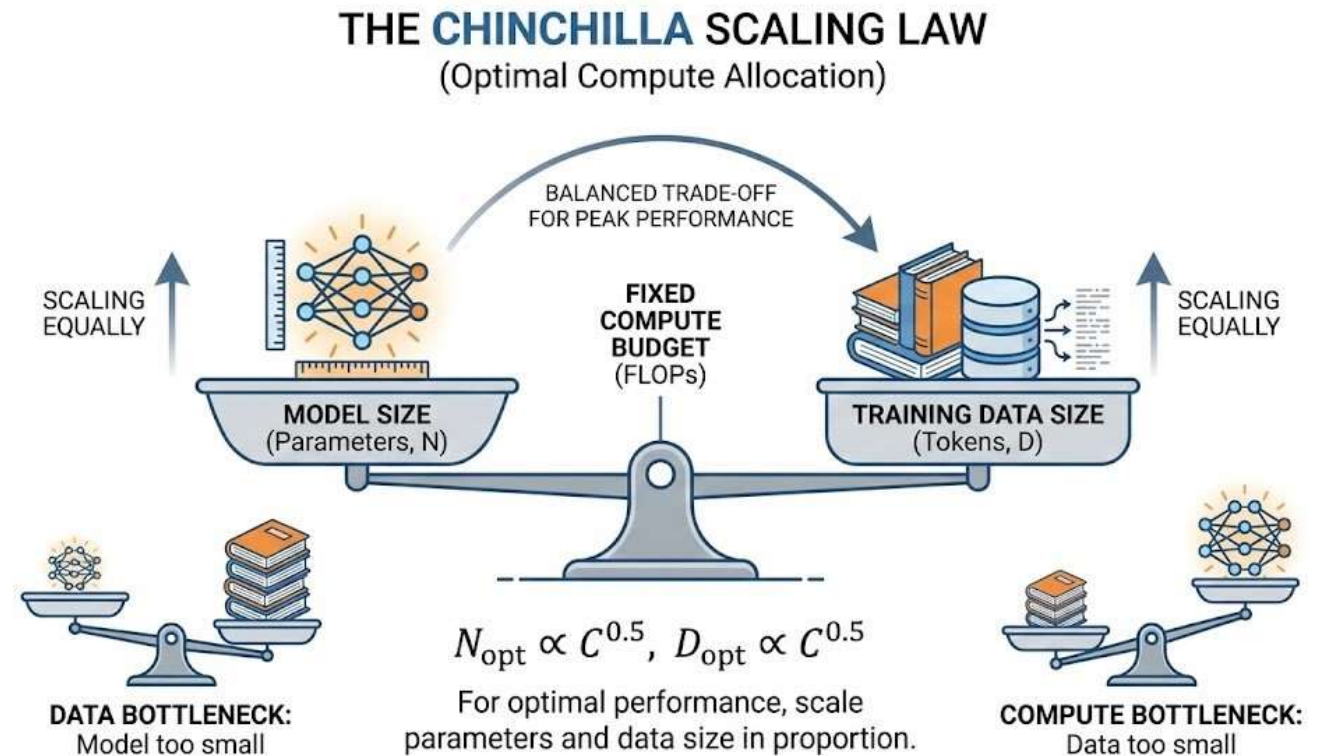
- They had capacity but hadn't seen enough tokens
- Increasing data while reducing model size (approximately) can achieve better performance under the same compute budget



CHINCHILLA LAW

TAKEAWAY

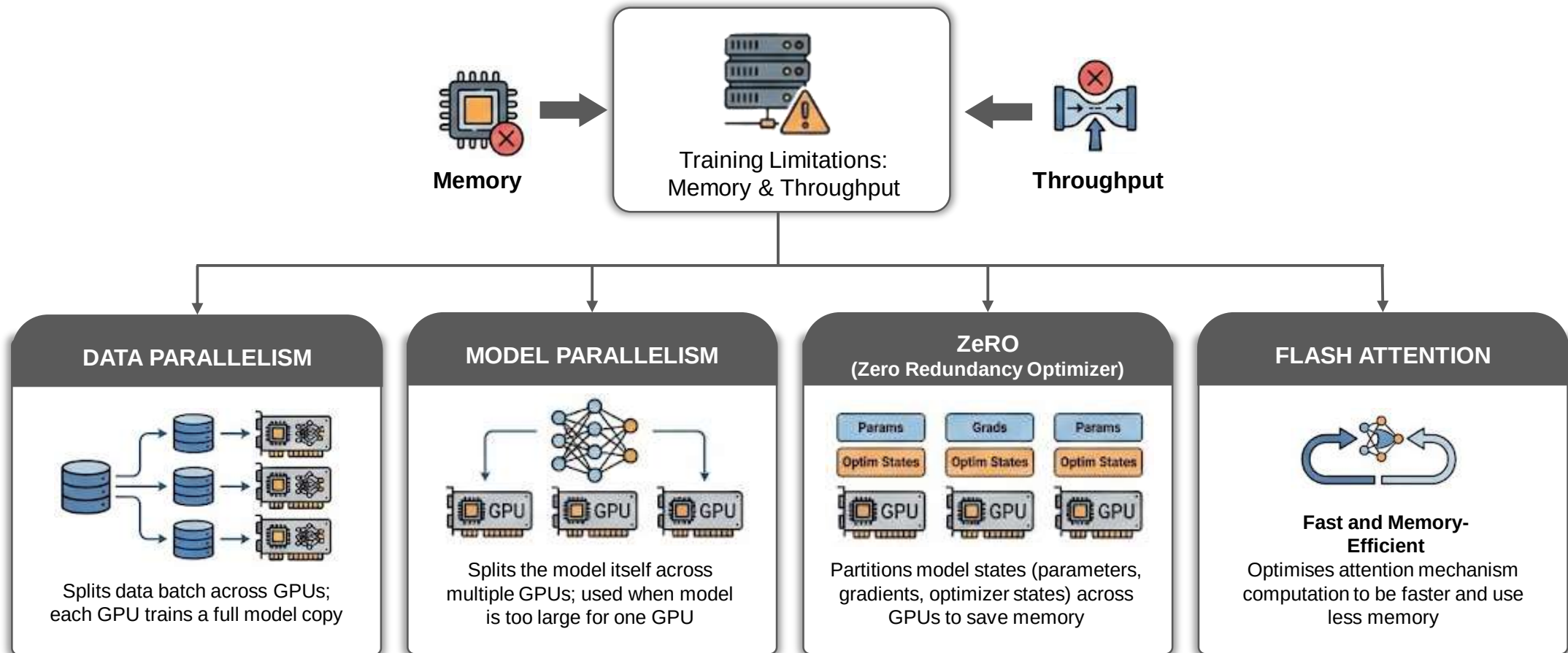
- Don't assume "bigger model = better"
- For a fixed budget, consider scaling data (and data quality) alongside or more than model size
- This also affects how we think about continued pretraining or SFT: more/better data can beat a slightly larger base model



Optimisation Techniques For LLM Training

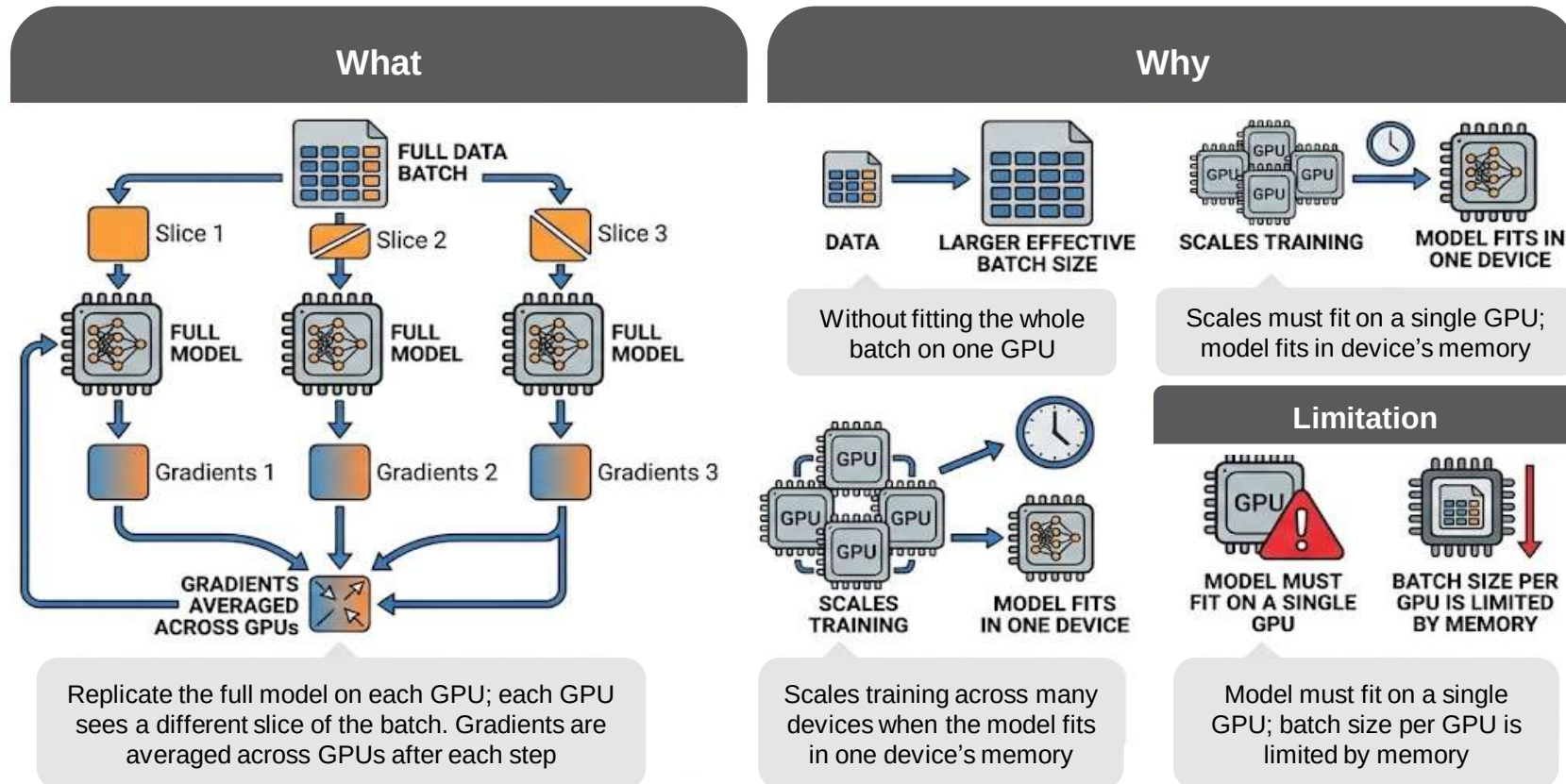
OPTIMISATION FOR LLM TRAINING

At scale, training is limited by **memory** and **throughput**. These system-level optimisations are standard in modern, large scale LLM training



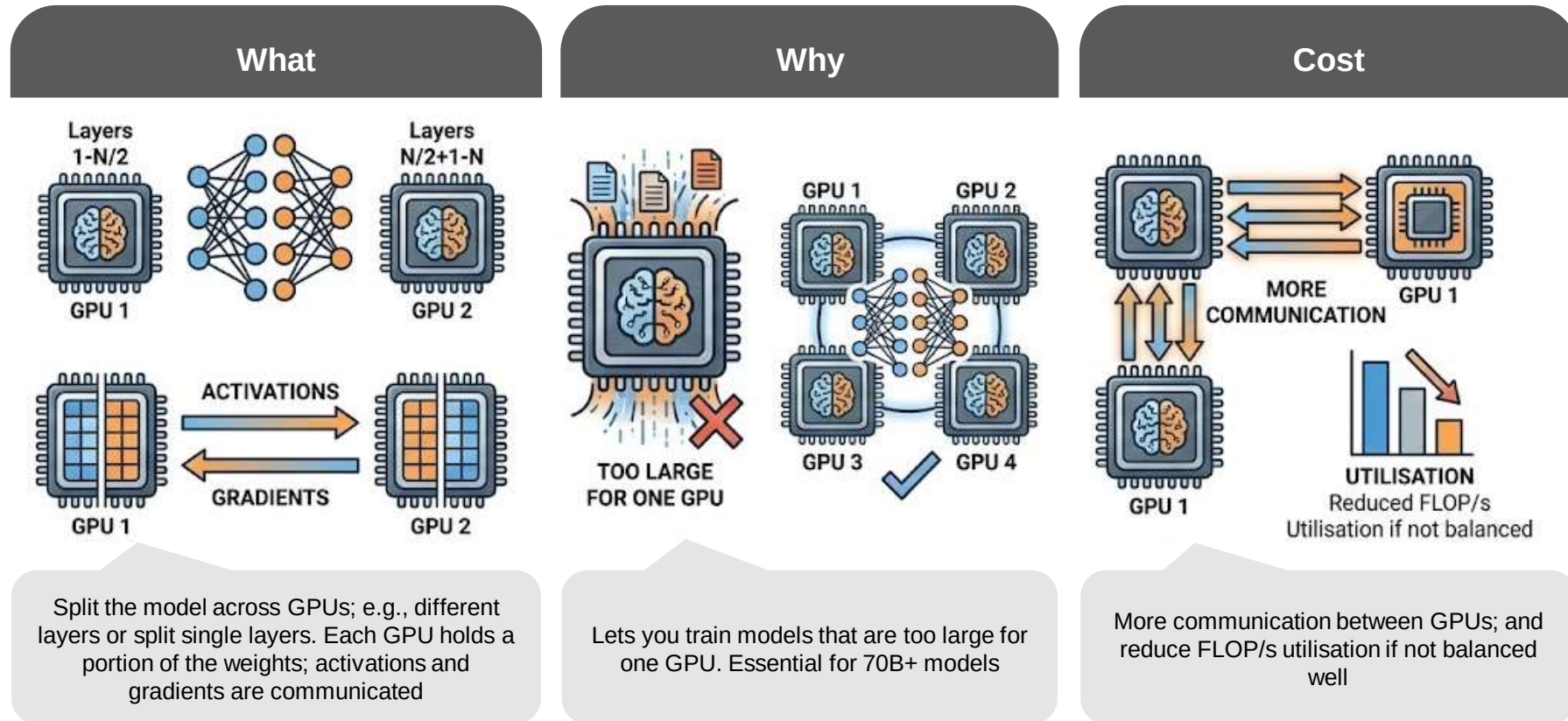
DATA PARALLELISM

- Data parallelism speeds up processing by replicating the entire model across multiple GPUs, where each GPU processes a different subset (mini-batch) of training data simultaneously
- It reduces total training time by distributing the workload, using synchronisation techniques like “all-reduce” to aggregate gradients and keep model weights consistent



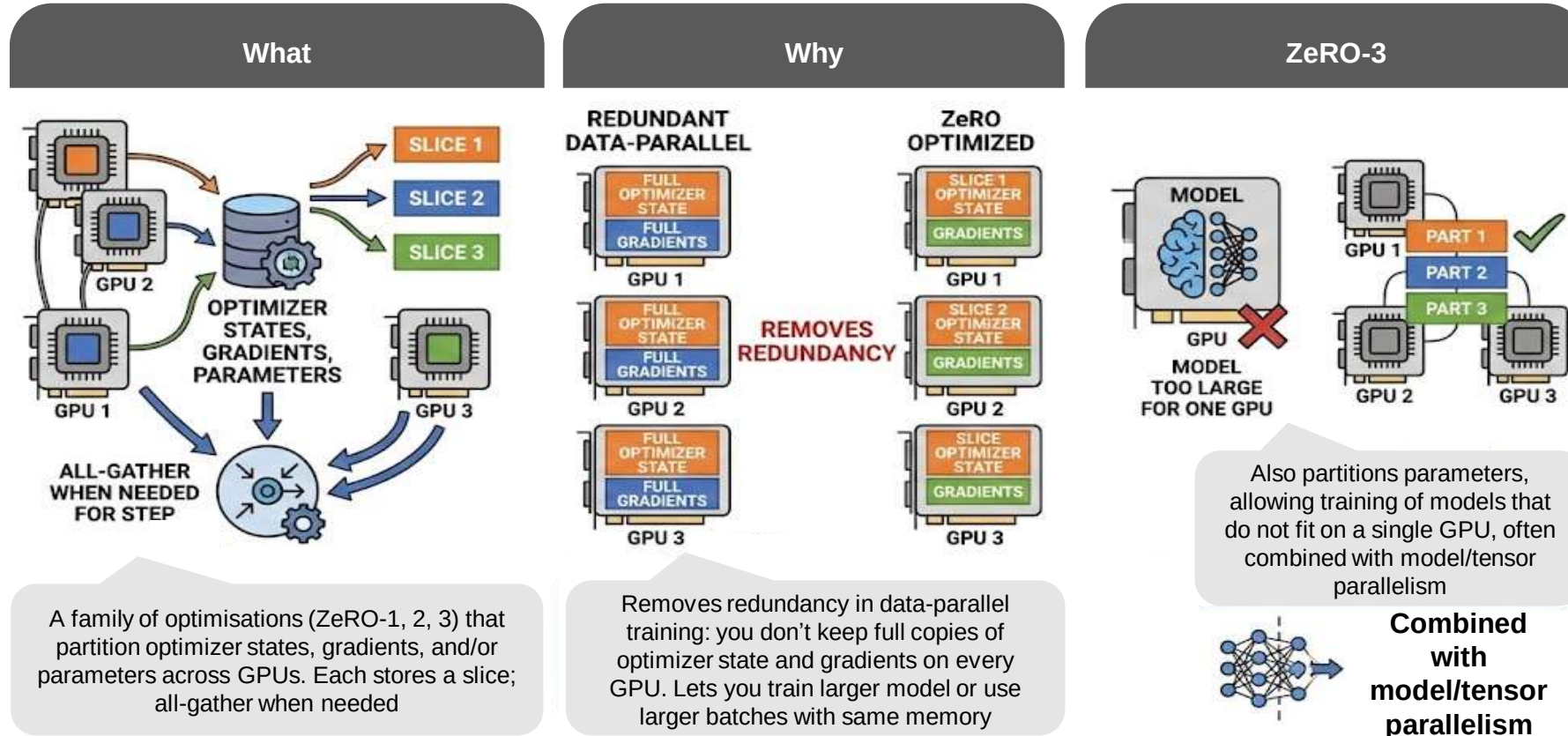
MODEL PARALLELISM

- Model parallelism is a distributed training technique for LLMs that splits a single model's parameters and layers across multiple GPUs, allowing for the training of models too large to fit into a single device's memory
- It enables, for example, massive transformer architectures (like GPT) to be partitioned across devices rather than fully replicating them, which is required when models exceed GPU memory capacity



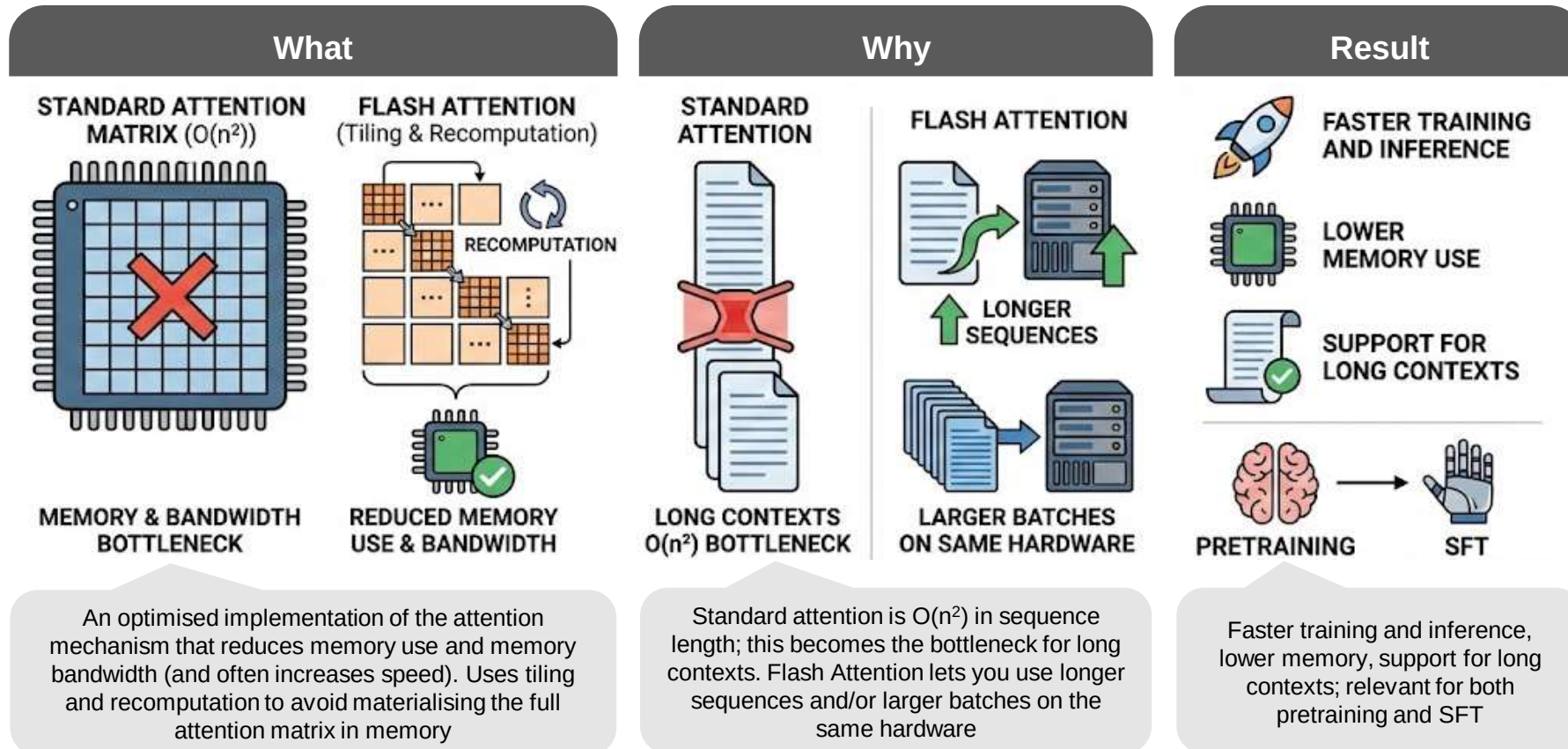
ZERO REDUNDANCY OPTIMIZER (ZeRO)

- The Zero Redundancy Optimizer (ZeRO) is a memory-optimisation technology developed by Microsoft for DeepSpeed that enables training LLMs with billions of parameters by partitioning model states (parameters, gradients, and optimiser states) across GPUs
- Unlike standard Data Parallelism, which replicates full model states on every device, ZeRO eliminates memory redundancy



FLASH ATTENTION

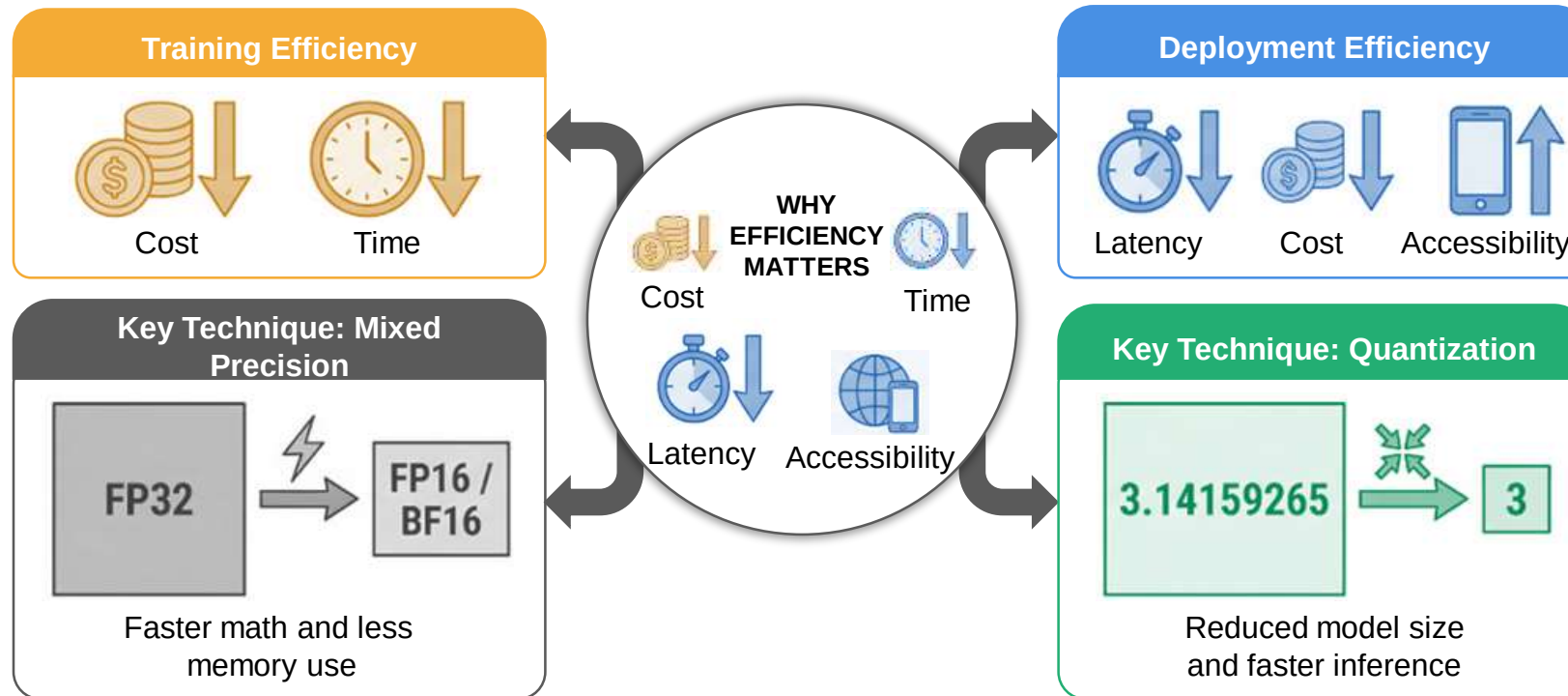
- Flash Attention is an I/O-aware, exact attention algorithm that accelerates LLM training by optimising how data moves between GPU memory levels (HBM and on-chip SRAM)
- Flash Attention improves attention efficiency by optimising memory access and computing attention in small blocks instead of storing the full attention matrix, enabling longer context lengths and faster training



LLMs: Efficiency

IMPORTANCE OF EFFICIENCY

- Efficiency matters both for **training** (cost, time) and **deployment** (latency, cost, accessibility)
- Key techniques: **quantization**, **mixed precision**, and a clear view of why efficiency matters



Quantization and **mixed precision** are the main levers for efficient deployment; some techniques (such as Flash Attention) improve both training and inference efficiency, while others (such as ZeRO) primarily optimise training

QUANTIZATION

Quantization store weights (and optionally activations) in lower precision; e.g. 8-bit or 4-bit integers instead of 16- or 32-bit floats

- We can dequantize on the fly during computation, or use integer kernels
- Quantization leads to smaller model size (e.g. 2× with INT8 vs FP16). This leads to less memory and bandwidth
- It enables running large models on consumer GPUs or smaller instances



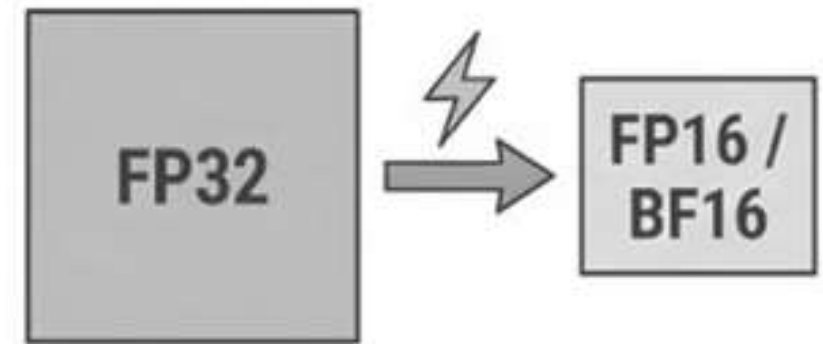
Reduced model size & faster inference

Quantization leads to a slight loss in numerical precision; for many models, 8-bit (and often 4-bit) is acceptable for inference and sometimes for training (quantized LoRA, etc.)

MIXED PRECISION

Mixed precision uses **FP16** or **BF16** for most forward and backward passes, and FP32 only where needed (e.g., optimiser states, loss scaling, or sensitive layers)

- Modern GPUs are much faster at 16-bit
- This method provides roughly 2x memory and often 2x speed optimisation as compared to full FP32, with minimal impact on training stability when done correctly (e.g. loss scaling)
- Inference is commonly done in FP16/BF16 or INT8/INT4
- Mixed precision requires techniques such as loss scaling to maintain numerical stability

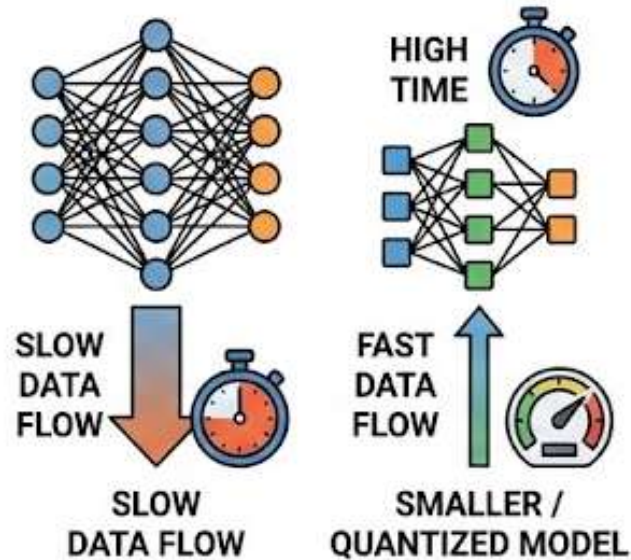


Faster math & less memory use

Mixed precision is the bridge from FP32 training to lower-precision deployment

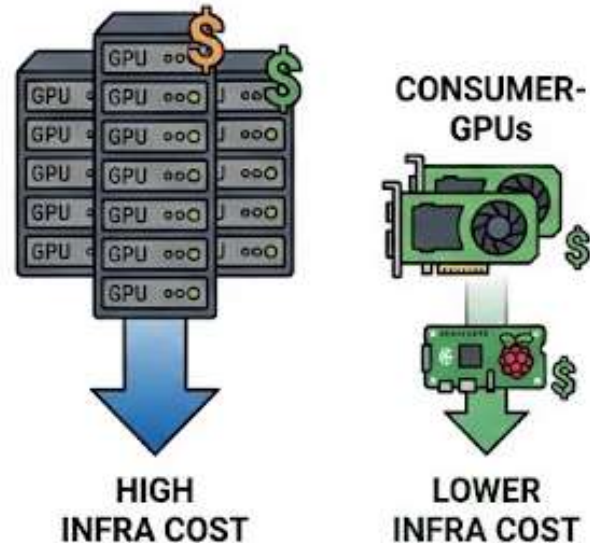
WHY EFFICIENCY MATTERS FOR DEPLOYMENT

Latency and Throughput



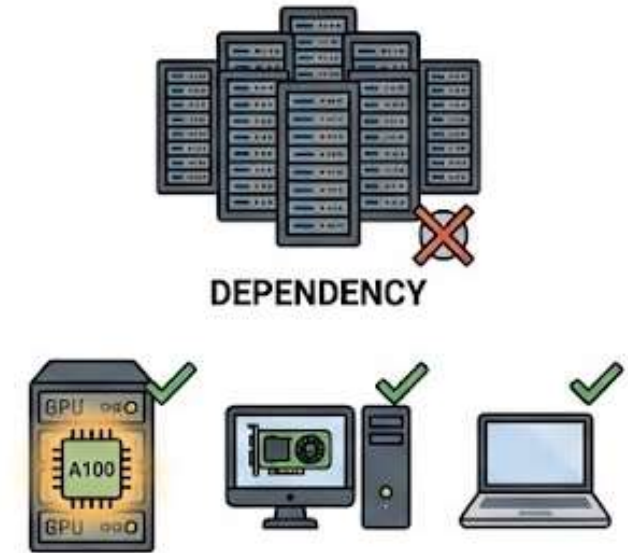
Smaller or quantized models need less memory bandwidth and compute per token → **Lower latency** and **higher throughput**

Cost Reduction



Fewer or smaller GPUs/instances per user → **lower infra cost**. Enables **edge** or **on-device deployment**

Accessibility

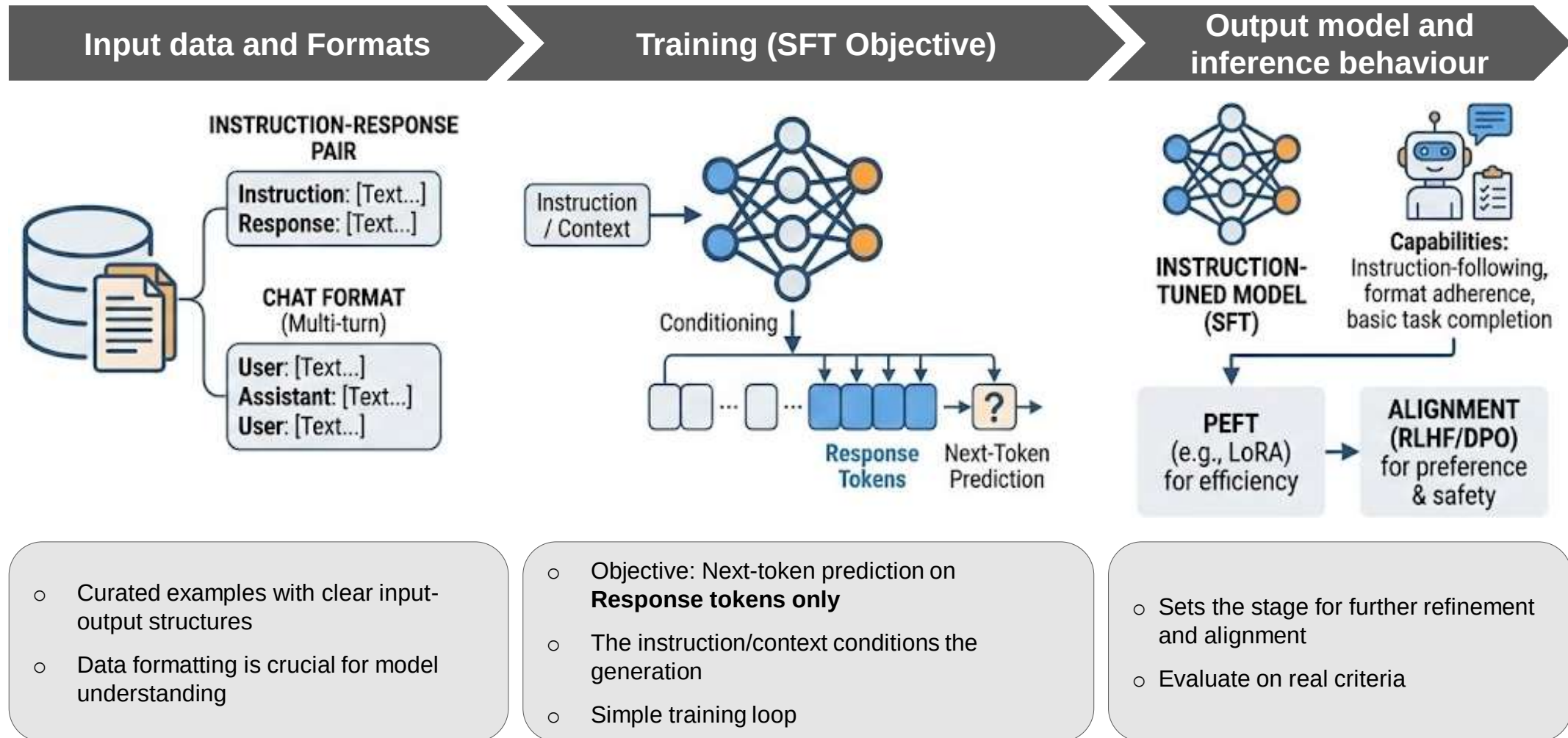


Efficient models can run on modest hardware (e.g. single A100, or consumer GPUs), **reducing dependency** on large clusters

Supervised Fine-Tuning

SUPERVISED FINE-TUNING

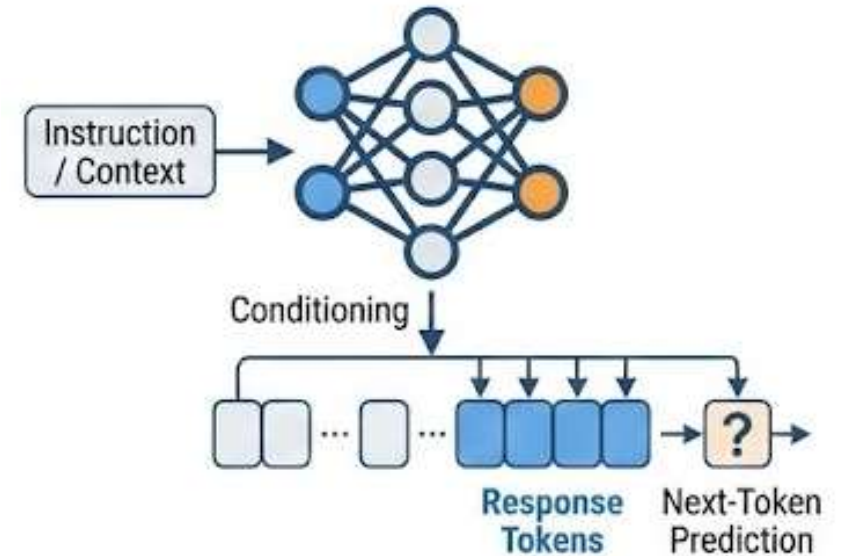
Supervised Fine-Tuning (SFT) is the first step that turns a base (or pretrained) model into an instruction-following assistant. SFT focuses on **objectives**, **data formats**, and **common workflow**



SFT OBJECTIVES

The primary objective of SFT is next-token prediction (language modelling) on the response only

- The model is given the instruction (and optionally context) and trained to maximise the likelihood of the target response
- Only the response tokens are used in the loss; the instruction/context tokens are not predicted (masked out in the loss)
- This teaches “given this input, produce this output” without wasting capacity on predicting the user’s text
- Some recipes also use a small weight on instruction tokens, or auxiliary losses (e.g. for length or format); the core is still a conditional LM on the assistant reply



DATA FORMATS IN SFT

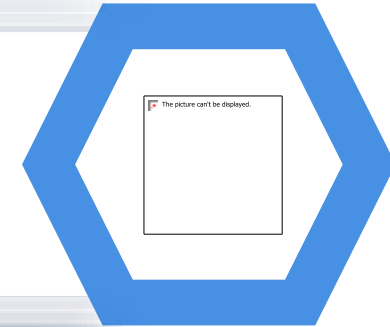


Instruction–response pairs

Each example has an instruction (or input) and a response (or output). Stored as JSON, JSONL, or similar

Conversation format

Multi-turn dialogues with roles (user, assistant, system). The model is trained to generate only the assistant turns; user and system messages are context. Aligns with chat APIs and real usage

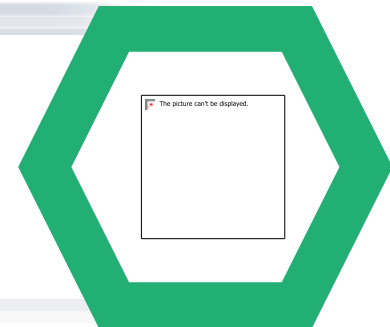


Single block with special tokens

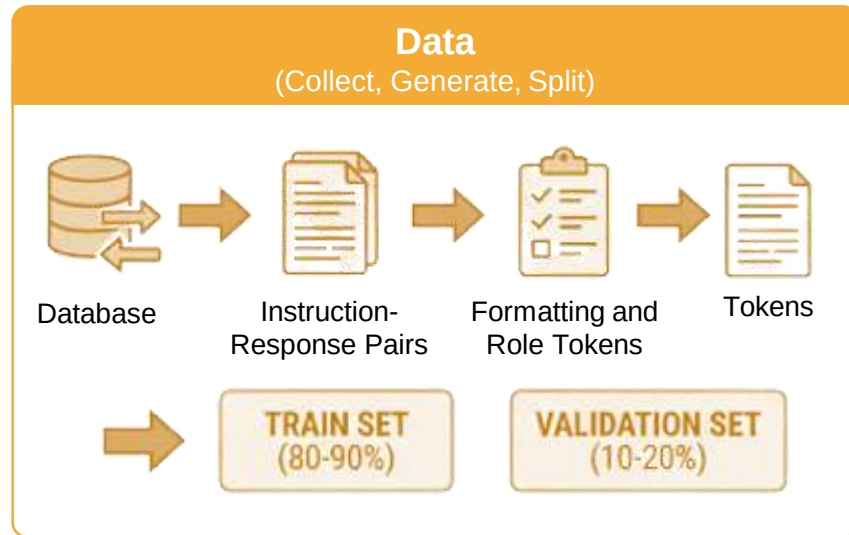
One text blob with <|user|>, <|assistant|>, etc. Loss is applied only on assistant segments. Common in pretrained tokenisers that support role tokens

Quality matters

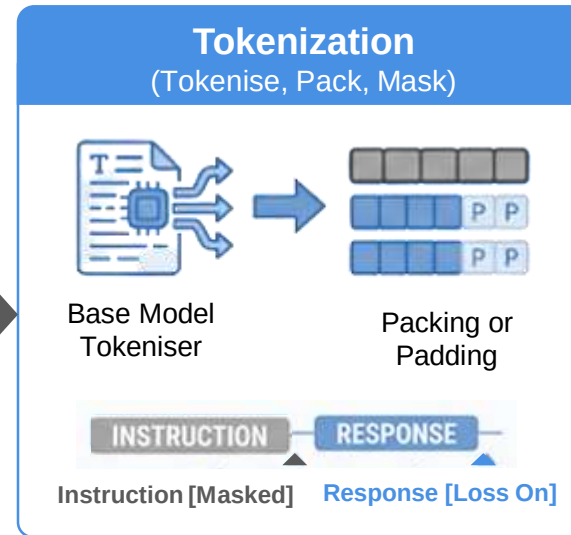
Noisy or inconsistent formats hurt SFT. Clean, diverse, and well-formatted data beats large but messy datasets



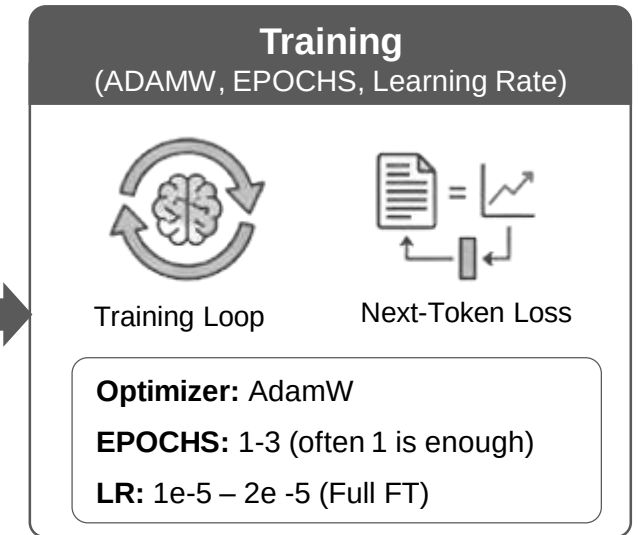
COMMON TRAINING WORKFLOW



Collect or generate (instruction, response) pairs; ensure consistent formatting and role tokens. Split into train/validation



Use the base model's tokeniser; pack or pad to fixed length or use packing for efficiency. Ensure loss masking only on response tokens



Train with next-token loss on response tokens; typical choices: AdamW or similar, Often 1 epoch is enough to avoid overfitting

